

sAIfer Lab

Joint lab on Safety and Security of AI

Liste

Creazione, indicizzazione, aggiunta ed eliminazione di elementi

Una lista raggruppa più oggetti in una sequenza

- Le liste sono sequenze ordinate e modificabili
- Si creano elencando gli oggetti tra parentesi quadre [ ], separati da virgole

```
# lista vuota
empty = []

# lista di numeri
numbers = [1, 2, 3, 100]

# lista di stringhe
superheroes = ["batman", "superman", "spiderman"]

# lista con elementi di tipo diverso
mixed_types = ["Hello World", [4, 5, 6], False]
```

Quattro modi per creare una lista

Lista vuota

```
a = []
# oppure
b = list()
```

Lista con elementi

```
a = [1, 2, 3]
# oppure
b = ['a', 'ciao', 3]
```

Da un'altra struttura dati

```
lista = list("ciao")
# ['c', 'i', 'a', 'o']
```

Con la funzione range

```
a = list(range(...))
# vedi la prossima slide
```

range() genera una sequenza di numeri interi

- Sintassi: `range(start, stop, step)`
- `start`: valore iniziale, incluso (default 0)
- `stop`: valore finale, escluso
- `step`: incremento tra due elementi (default 1)
- Un oggetto `range` non è una lista: conserva solo start, stop e step
- `list()` lo converte in una lista di interi

range(): tre esempi

```
a1 = list(range(10))
# start == 0, stop == 10 (escluso), step == 1
# [0, 1, 2, 3, 4, 5, 6, 7, 8, 9]

a2 = list(range(3, 10))
# start == 3, stop == 10 (escluso), step == 1
# [3, 4, 5, 6, 7, 8, 9]

a3 = list(range(3, 10, 2))
# start == 3, stop == 10 (escluso), step == 2
# [3, 5, 7, 9]
```

Documentazione: docs.python.org/3/library/stdtypes.html#range

Ogni elemento ha un indice di posizione

- Una lista è una collection: si gestisce come blocco unico o un elemento alla volta
- Gli elementi possono essere di tipo diverso: interi, float, stringhe, altre liste
- L'indice vale 0 per il primo elemento e len(lista) - 1 per l'ultimo
- Con l'indice si legge o si modifica un elemento
- Un indice inesistente produce un errore:
  `IndexError: list index out of range`

L'indice legge e modifica gli elementi

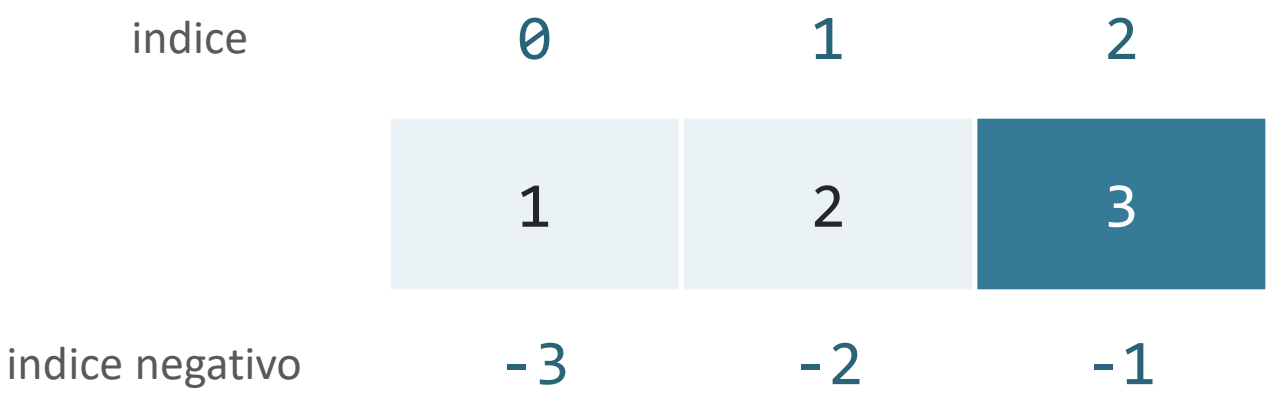
```
a = [1,2,3]
i = 0
print(a[i])          # primo elemento
print(a[len(a)-1])   # ultimo elemento
print(a[8])          # IndexError: list index out of range

i = 1
# modifico il secondo elemento 'in place',
# cioè sulla lista stessa, senza crearne una nuova
a[i] = 10
print(a[i])          # stampo il secondo elemento

print(a[5])
# IndexError: list index out of range
# NB anche print(a[3]) produce lo stesso errore: perché?
```

Gli indici negativi contano dalla fine

L'ultimo elemento ha indice -1, il penultimo -2 e così via




```
a = [1, 2, 3]
print(a[-1])  # 3: ultimo elemento, equivale a a[len(a)-1]
print(a[-2])  # 2: penultimo elemento
```

Scorrere una lista con un ciclo while

In avanti: l'indice va da 0 a `len(a) - 1`

```
a = [10, 20, 30]
i = 0
while i < len(a):
    print(a[i])  # 10, poi 20, poi 30
    i = i + 1
```

All'indietro: da `len(a) - 1` fino a 0

```
a = [10, 20, 30]
i = len(a) - 1
while i >= 0:
    print(a[i])  # 30, poi 20, poi 10
    i = i - 1
```

Confronti tra elementi

Per verificare se gli elementi di una lista sono ordinati secondo un certo criterio (per es., in senso crescente), è necessario confrontare coppie di elementi adiacenti

```
a = [1, 2, 3, 4]
ordinati = True  # si assume che siano ordinati

i = 0  # indice del primo elemento di ogni coppia...
while i < len(a) - 1  # ...fino al penultimo elemento
    if a[i] > a[i + 1]:  # non sono ordinati
        ordinati = False
    i = i + 1

print(ordinati)  # True se sono ordinati, False altrimenti
                 # (in questo esempio, True)
```

a + b crea una nuova lista

- `c = a + b` concatena gli elementi in una nuova lista: `a` e `b` non cambiano

```
a = [1, 2, 3, 4]
b = [5, 6]
c = a + b
print(c) # [1, 2, 3, 4, 5, 6]

a = [1, 2, 3, 4]
b = [5]
c = a + b
print(c) # [1, 2, 3, 4, 5]

a = []
b = [4]
c = a + b
print(c) # [4]
```

Costruzione di una lista per concatenazione

- Si parte da una lista vuota, si aggiunge un elemento alla volta (si assume che gli elementi siano numeri e che vengano acquisiti mediante `input`)

```
n = int(input("Quanti elementi? "))
lista = []

i = 0
while i < n:
    elemento = float(input("  prossimo elemento: "))
    lista = lista + [elemento]
    i = i + 1

print(lista)
```

Una lista può contenere altre liste

- In `a = [1, [10, 20, 30], 3]` il secondo elemento `a[1]` è la lista `[10, 20, 30]`
- L'elemento interno si indicizza come una normale lista: `a[1][2]` è l'elemento di indice 2 di `[10, 20, 30]`

```
a = [1, [10, 20, 30], 3]
print(a[1])     # [10, 20, 30]
print(a[1][0])  # 10
b = a[1][2]
print(b)        # 30
```

Operatori relazionali == e !=

Due liste sono uguali se contengono gli stessi elementi nello stesso ordine

```
l1 = [1, 2, 3]
l2 = [1, 2, 3]     # stessa sequenza di valori di l1
l3 = [3, 1, 2]     # stessi valori, ordine diverso
l4 = [1, 1, 2, 3]  # stessi valori, uno compare più volte
print(l1 == l2)    # True
print(l1 == l3)    # False
print(l1 != l2)    # False
print(l1 != l3)    # True
print(l1 == l4)    # False
```

Lo slicing estrae una sottolista

- Sintassi: `lista[start:stop:step]`
- `start`: indice iniziale, incluso
- `stop`: indice finale, escluso
- `step`: numero di elementi da saltare a ogni passo
- Gli indici sono facoltativi. Default: `start` 0, `stop` lunghezza della lista, `step` 1

lista[2:10:3] seleziona gli indici 2, 5 e 8

`lista[2:10]`: dall'indice 2 all'indice 10, escluso



`lista[2:10:3]`: stesso intervallo, con step 3



Il risultato contiene `lista[2]`, `lista[5]` e `lista[8]`

I numeri nelle caselle sono gli indici di una lista di 12 elementi

Slicing: i casi più comuni

```
lista2 = lista[2:10:3]  # da 2 a 10 (escluso), step 3
lista2 = lista[2:10]    # da 2 a 10 (escluso), step 1
lista2 = lista[2:]      # da 2 alla fine, step 1
lista2 = lista[:10]     # dall'inizio a 10 (escluso)

# tutti gli elementi della lista: omettendo start,
# stop e step, questi assumono i valori di default
lista2 = lista[:]

lista2 = lista[10:2:-1] # da 10 a 2 (escluso), step -1
lista2 = lista[::-1]    # tutti, in ordine inverso
```

Tabella riassuntiva dello slicing (1/2)

Sintassi	Descrizione	Esempio / Note
`lista[start:stop]`	Da start a stop (escluso)	lista[2:5] → pos. 2, 3, 4
`lista[start:]`	Da start fino alla fine	lista[2:] → dall'indice 2 (3° elemento) in poi
`lista[:stop]`	Dall'inizio fino a stop (escluso)	lista[:4] → primi 4 elementi
`lista[:]`	Copia completa della lista	
`lista[start:stop:step]`	Da start a stop (escluso), saltando di step elementi	lista[1:7:2] → pos. 1, 3, 5

Tabella riassuntiva dello slicing (2/2)

Sintassi	Descrizione	Esempio / Note
`lista[::step]`	Tutta la lista con passo step	lista[::2] → un elemento ogni due
`lista[::-1]`	La lista al contrario	Inversione rapida
`lista[start::-1]`	Da start all'inizio, all'indietro	lista[4::-1] → da pos. 4 a 0
`lista[:stop:-1]`	Dalla fine fino a stop (escluso), all'indietro	lista[:2:-1] → da fine a pos. 3
`lista[start:stop:-1]`	Da start a stop (escluso), all'indietro	lista[5:1:-1] → pos. 5, 4, 3, 2

Inserimento di un valore in una lista

Mediante la concatenazione si può inserire direttamente un valore all'inizio o alla fine di una lista

```
a = [1, 2, 3, 4, 5, 6]
a = a + [7] # [1, 2, 3, 4, 5, 6, 7]
a = [0] + a
print(a)    # [0, 1, 2, 3, 4, 5, 6, 7]
```

Per inserire un valore in una qualsiasi altra posizione si possono combinare concatenazione e slicing

```
a = [1, 2, 3, 4, 5, 6]
x = 0     # valore da inserire
i = 3     # indice del nuovo valore (quarta posizione)
a = a[:i] + [x] + a[i:]
print(a)  # [1, 2, 3, 0, 4, 5, 6, 7]
```

Eliminazione di un elemento in una data posizione

Anche questa operazione può essere eseguita mediante la combinazione di concatenazione e slicing

```
a = [6, -2, -8, 4, 1, -11]
i = 2     # indice dell'elemento da eliminare (il terzo)
a = a[:i] + a[i + 1:]
print(a)  # [6, -2, 4, 1, -11]
```

Selezione di elementi in base a una condizione

Per selezionare un sottoinsieme degli elementi di una lista in base ai loro valori, si possono combinare iterazione e istruzione condizionale. Esempio: selezionare i soli elementi positivi, memorizzandoli in una nuova lista

```
a = [6, -2, -8, 4, 1, -11]
i = 0
b = []

while i < len(a):
    if a[i] > 0:
        b = b + [a[i]]
    i = i + 1

print(b)  # [6, 4, 1]
```

Operatori in e not in

Per verificare se un valore è presente tra gli elementi di una lista si possono usare gli operatori `in` e `not in`

```
a = [1, 2, 3, 4, 5, 6]
3 in a        # True
0 in a        # False
4 not in a    # False
10 not in a   # True
[2, 3] in a   # False: si considerano singoli elementi,
              # non sottosequenze di elementi
```

Alcune funzioni che si applicano alle liste

Funzione	Descrizione	Esempio
`len(L)`	Numero di elementi	`len(L)`
`sum(L)`	Somma degli elementi numerici	`sum([1,2,3])`
`min(L)`	Valore minimo	`min([1,2,3])`
`max(L)`	Valore massimo	`max([1,2,3])`
`sorted(L)`	Restituisce una nuova lista ordinata; L non cambia	`sorted([3,1,2])` `# [1, 2, 3]`



sAIfer Lab

Joint lab on Safety and Security of AI

Riepilogo ed esercizi

Le operazioni principali sulle liste

Riepilogo

- Saper iterare su una lista con `while`: in avanti, all'indietro, con passo variabile
- Saper aggiungere elementi e costruire nuove liste a partire da quelle esistenti
- Saper selezionare o eliminare elementi in base a condizioni sul valore (es. numeri negativi) o sull'indice (es. posizioni pari): non confondere le due cose
- Gestire i casi limite: il primo non ha precedente, l'ultimo non ha successivo

Esercizi

- Data una lista composta da numeri, contare gli elementi positivi (es.: `a = [5, -2, 4, 7, -9, 6]`)
- Considerare una lista contenente altre liste:

  ```
  a = [1, 2, ['a', 'b', 'c']]
  ```

  – cosa succede con `a[2] = 10`?

  – e con `a[2][1] = 100`?

  – `a[2]` è a sua volta una lista: cosa succede con `a[2] = a[2] + [100]`?